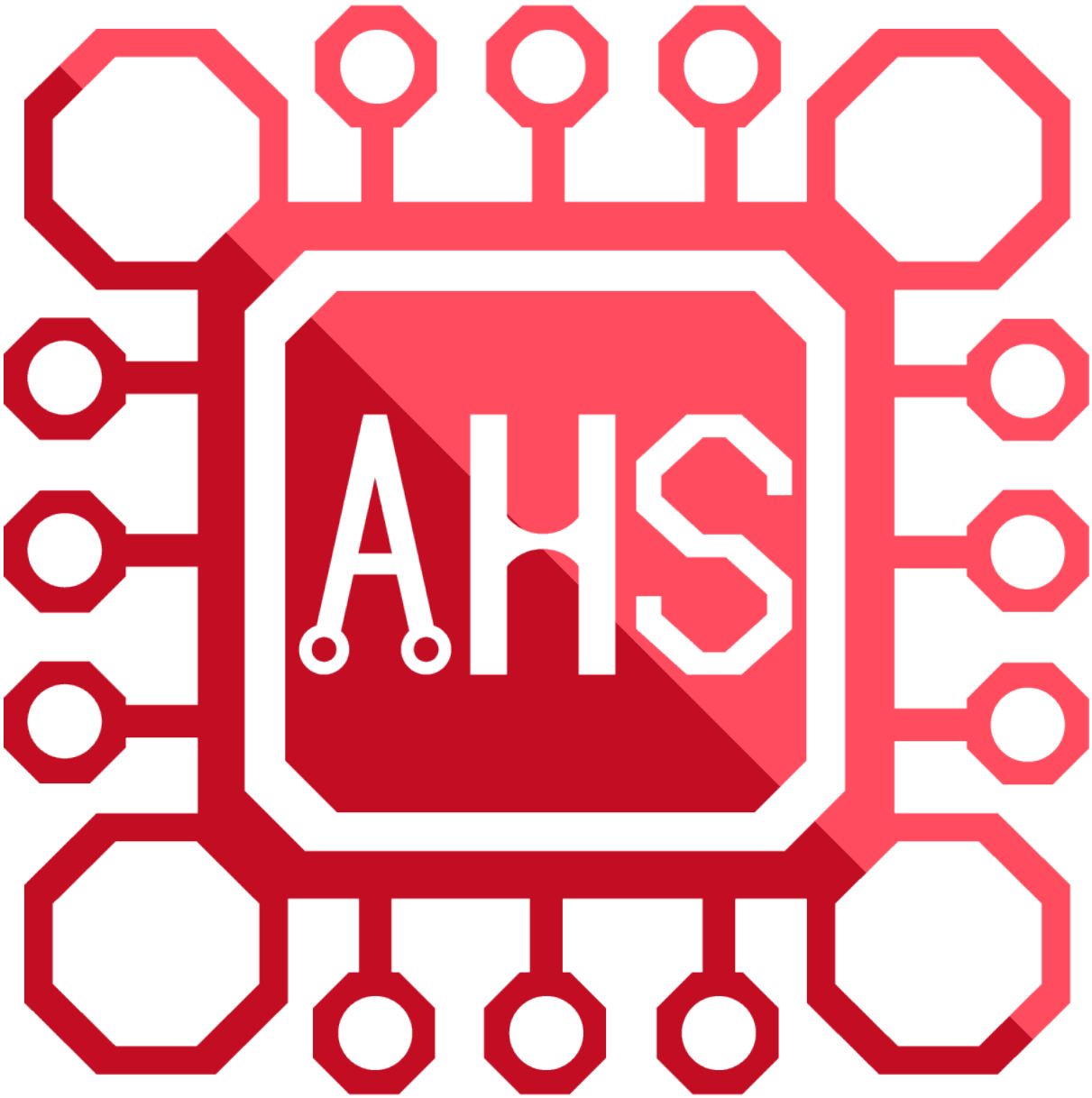


tutorial-aspdac2025 Documentation

Release 1.2024-preview

Massimo DiPierro

1 Overview **3**

2 Setup **7**

3 Schedule **9**

4 Organizers **11**

5 Repos and Publications **13**
5.1 Handout 13
5.2 Tutorial Instructions 14

Note Important Update!

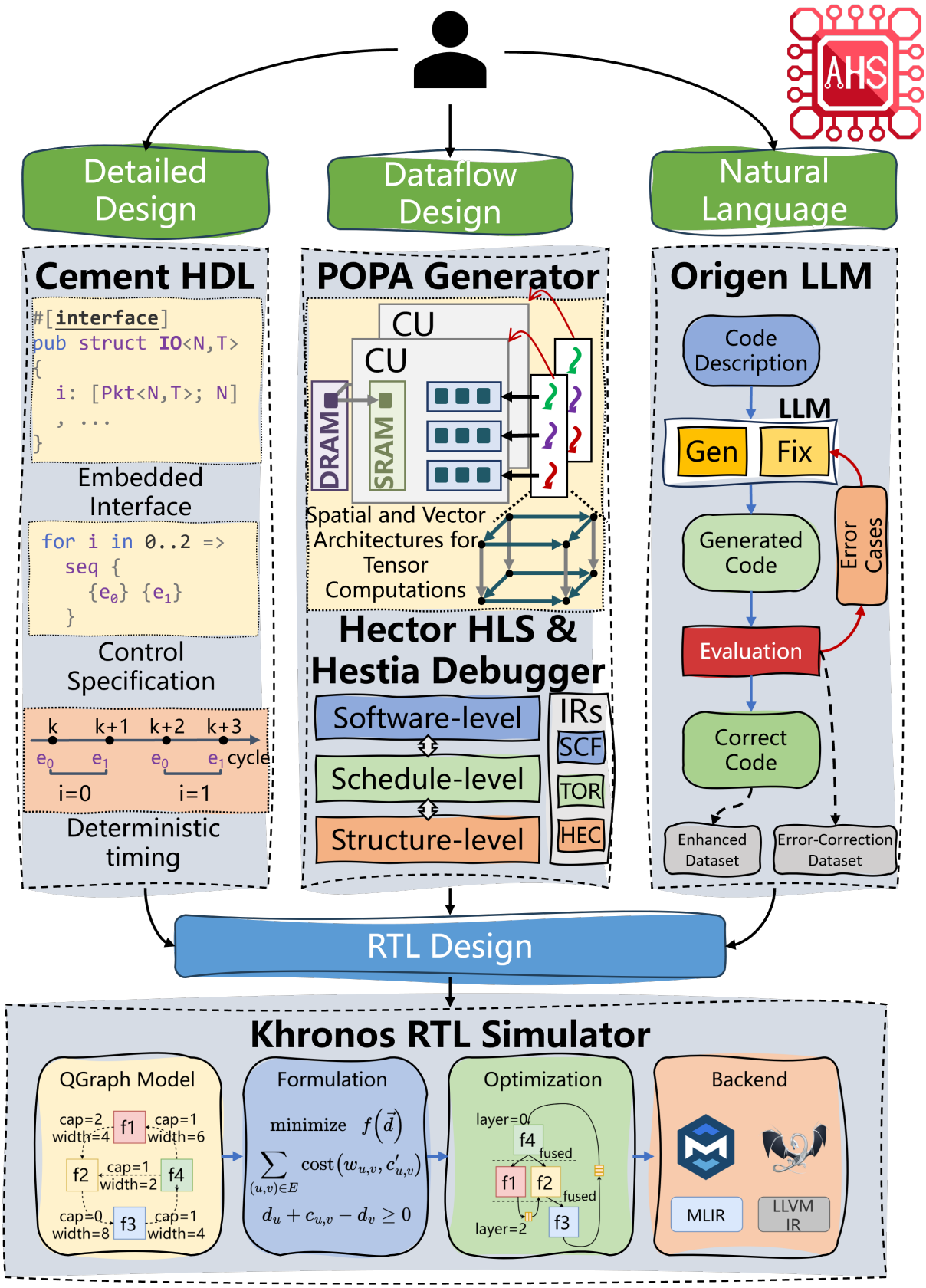
We have uploaded the handout materials for the tutorial, including environment setup instructions and the reading materials for the frameworks to be introduced in the tutorial. Please refer to Section 5.1.1 for more details.

If you have any questions, especially about environment setup, please contact Youwei Xiao (shallwe@pku.edu.cn). We will try our best to help you.

Chapter 1

Overview

Compared to software design, hardware design is more expensive and time-consuming. This is partly because software community has developed a rich set of modern tools to help software programmers to get projects started and iterated easily and quickly. However, the tools are seriously antiquated and lacking for hardware design. Modern digital chips are still designed manually using hardware description language such as Verilog or VHDL, which requires low-level and tedious programming, debugging, and tuning. In this tutorial, we will introduce Agile Hardware Specialization (AHS) : A toolbox for Agile Chip Front-end Design.



The tutorial will highlight the methodology and open source tools in AHS for both chip design and verification. From the design perspective, AHS present three ways that use different programming

interfaces and target different scenarios, including; 1) a multi-level hardware intermediate representation based high-level synthesis flow, which uses C and C++ as the programming language; 2) an embedded hardware description language, which uses Rust as the programming language; 3) a large language model (LLM)-powered hardware design flow, which uses natural language as the programming language. These three different methodologies exhibit different trade-offs in productivity and PPA (performance, power, and area) for chip design. From the verification perspective, we will present agile simulation and debugging tools, which can check the functional and performance behaviors of the hardware. The attendees will learn the methodology, design automation fundamentals, and software tools of AHS.

Chapter 2

Setup

We provide both Docker and local setup. See Environment Setup page for details.

Chapter 3

Schedule

Chapter 3

Chapter 4

Organizers

Chapter 4

Chapter 5

Repos and Publications

Framework	Repository	Materials
POPA	pku-liang/popa	FPGA'24 Paper, FCCM'23 Paper
Hector	pku-liang/Hector	ICCAD'22 Paper
Hestia	pku-liang/hestia	MICRO'24 Paper
Cement	pku-liang/Cement	FPGA'24 Paper, Crates.io Package, Documentation
OriGen	pku-liang/OriGen	ICCAD'24 Paper, Hugging Face
Khronos (ksim)	pku-liang/ksim	MICRO'23 Paper

5.1 Handout

5.1.1 Handout Materials

We provide handout materials for the tutorial. It includes environment setup instructions and the reading materials for the frameworks to be introduced in the tutorial.

If you have any questions, especially about environment setup, please contact Youwei Xiao (shallwe@pku.edu.cn). We will try our best to help you.

Environment Setup

We provide two setup methods for the involved frameworks: Docker and local installation. We recommend using Docker to try out the frameworks.

Docker

Easiest way: use the built Docker image.

We provide a built Docker image for the tutorial. You can pull it from Docker Hub:

```
docker pull uvxiao/ahs-tutorial:aspdac2026
```

Then, you can run the following command to start the container:

```
docker run -it --rm uvxiao/ahs-tutorial:aspdac2025
```

The frameworks are installed inside the `/root/repos` directory.

Building Docker Image from Source

We also provide a repository for building the Docker image from source. Please refer to the ahs-docker repository for details. Here are some takeaways instructions:

```
git clone git@github.com:pku-liang/ahs-docker.git
cd ahs-docker
```

```
# clone the framework collection
git clone git@github.com:pku-liang/ahs-tutorial.git repos --recursive
# make the run.sh executable
chmod +x run.sh
# pull the ubuntu images
./run.sh pull
# build the base Docker image
./run.sh build -i ahs-docker
# run a container with the frameworks mounted
./run.sh run -i ahs-docker -c ahs-docker -v mount.json
docker attach ahs-docker
# then inside the container:
cd /root/repos
chmod 777 ./install.sh
./install.sh
```

The produced Docker image should be the same as the `uvxiao/ahs-tutorial:aspdac2025` image.

Local Installation

We also provide a local installation script for the tutorial. Some prerequisites, including but not limited to Ninja, gperf, zip, are required, please refer to Dockerfile as a reference.

Then, you can run the following commands to install the frameworks:

```
git clone git@github.com:arch-of-shadow/ahs-tutorial.git
cd ahs-tutorial
git submodule update --init
chmod 777 install.sh
./install.sh
```

Reading Materials

For every framework to be introduced in the tutorial, we provide its open-source repository link and related reading materials like publications, documentation, etc.

Framework	Repository	Materials
POPA	pku-liang/popa	FPGA'24 Paper(Lasa), FCCM'23 Paper
Hector	pku-liang/Hector	ICCAD'22 Paper
Hestia	pku-liang/hestia	MICRO'24 Paper
Cement	pku-liang/Cement	FPGA'24 Paper, Crates.io Package, Documentation
OriGen	pku-liang/OriGen	ICCAD'24 Paper, Hugging Face
Khronos (ksim)	pku-liang/ksim	MICRO'23 Paper

Feel free to have a glance at the materials to get a sense of our works! We'll assist you to use them step-by-step in the tutorial.

5.2 Tutorial Instructions

5.2.1 Cement instructions

Introductions on the **cmtrs** embedded language, the **cmtc** compiler and some examples will be presented at the tutorial.

Please read the documentation for the basic concepts and language usage.

Cement Example Contents

The examples are in `Cement\crates\cmtc\examples` directory.

We will demonstrate these examples * `fir.rs` Finite Impulse Response filter.

- `fn fir3_3()` shows how to write an FIR with fixed length 3.
- `fn gen_fir()` shows how to generate FIR with arbitrary length.
- `fn gen_fir_addertree()` further implements the FIR with an adder tree, which demonstrates the use of `#[gen_fn]` to generate hardware recursively.
- `fn make_tb()` demonstrates how to write a testbench in `cmtrs`.

- `gemm.rs` General Matrix Multiplication
  - `fn mac()` A multiply-accumulate unit to be used in the GeMM example.
  - `fn gemm()` A GeMM unit with only one MAC, which is controlled by a three-level for loop FSM.
  - `fn tb()` A testbench for gemm.
- `gemm_unrolled.rs` Unrolled gemm
  - `fn mac()` A multiply-accumulate unit to be used in the GeMM example.
  - `fn gemm_unrolled()` A GeMM unit with `#factor` MACs, the innermost loop is unrolled and the memories are partitioned.
  - `fn tb()` A testbench for gemm_unrolled.

Running Cement Examples

Steps to run an example

```
cd Cement
cargo run -p cmtc --example <name_of_example>
```

Results:

- A generated System Verilog file at the root directory
- A Khronos simulation environment in `tb` directory

Simulation with Khronos

After running an example

```
cd tb/<name_of_example>
make all
./<executable_name>
```

Results:

- The executable will simulate the generated hardware, and prints the `sim_print!` messages in the testbench.